EXPERIMENT-1

AIM: Implement multilayer perceptron algorithm for MNIST Handwritten Digit Classification.

DESCRIPTION:

One of your assignments is to implement a backpropagation algorithm on a simple MLP architecture. Now, based on that simple BPNN architecture, develop a new model of BPNN capable of tackling a general digit recognition task. Once again, please employ digit datasets taken from sklearn import datasets and split into training (80%) and validation (20%) datasets. Since the size of the sample image is 8 x 8 pixels, so the architecture of your new BPNN model has to be adjusted accordingly. Your python notebook must include the computation of

(A) training accuracy and validation accuracy curves, and

(B) performance analysis of the model based on the associated confusion matrix

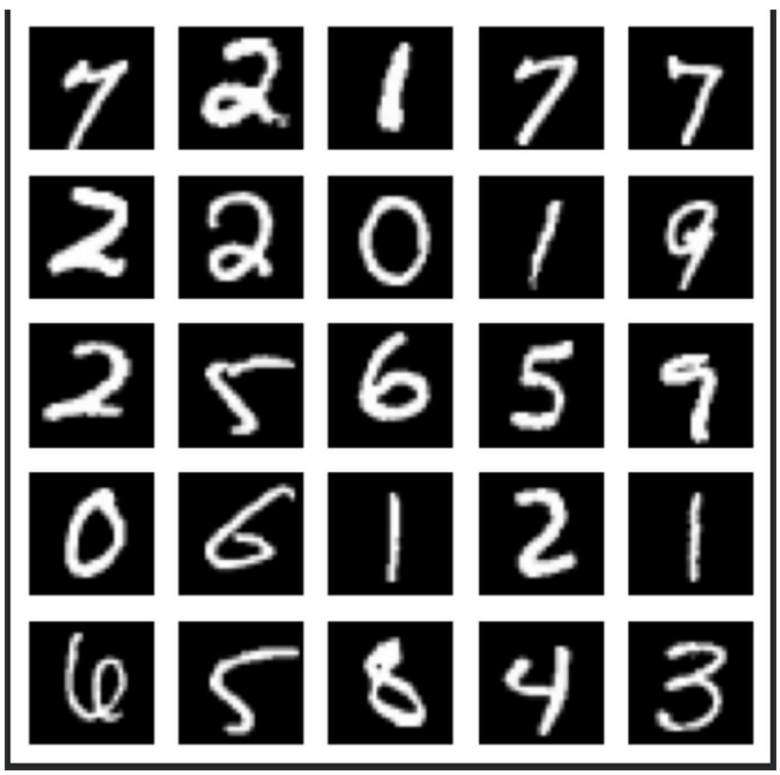
PROGRAM:

```
import numpy as np
from matplotlib import pyplot as plt
from tensorflow.keras.datasets import mnist

# Load MNIST dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Count training labels
unique, counts = np.unique(y_train, return_counts=True)
print("Train:", dict(zip(unique, counts)))

# Count test labels
unique, counts = np.unique(y_test, return_counts=True)
print("\nTest labels:", dict(zip(unique, counts)))

# Select 25 random images
indexes = np.random.randint(0, x_train.shape[0], size=25)
images = x_train[indexes]
labels = y_train[indexes]

# Plot images
plt.figure(figsize=(5, 5))

for i in range(len(indexes)):
    plt.subplot(5, 5, i + 1)
    image = images[i]
    plt.imshow(image, cmap="gray")
    plt.axis("off")
```

```
# Save and show figure

plt.savefig("mnist_sample.png")
plt.show()

# Close all figures
plt.close("all")
```

OUTPUT:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Train: {np.uint8(0): np.int64(5923), np.uint8(1): np.int64(6742), np.uint8(2): np.int64(5958),
 np.uint8(3): np.int64(6131), np.uint8(4): np.int64(5842), np.uint8(5): np.int64(5421), np.uint8(6):
 np.int64(5918), np.uint8(7): np.int64(6265), np.uint8(8): np.int64(5851), np.uint8(9):
 np.int64(5949)}

Test labels: {np.uint8(0): np.int64(980), np.uint8(1): np.int64(1135), np.uint8(2): np.int64(1032),
 np.uint8(3): np.int64(1010), np.uint8(4): np.int64(982), np.uint8(5): np.int64(892), np.uint8(6):
 np.int64(958), np.uint8(7): np.int64(1028), np.uint8(8): np.int64(974), np.uint8(9): np.int64(1009)}
```



EXPERIMENT-4

AIM: Design a neural network for predicting house prices using Boston Housing Price dataset.

DESCRIPTION:

This project aims to predict the median value of owner-occupied homes in the Boston area using deep learning techniques. The model is trained on the Boston Housing dataset, which consists of various features such as crime rate, average number of rooms, and accessibility to radial highways. This project demonstrates the effectiveness of deep learning in solving regression problems, specifically in the case of predicting housing prices. Further improvements can be made by exploring different neural network architectures and hyperparameter tuning.

PROGRAM:

```
# Step 1: Import required libraries

import numpy as np

import tensorflow as tf

from tensorflow import keras

from tensorflow.keras import layers

from tensorflow.keras.datasets import boston_housing

import matplotlib.pyplot as plt


# Step 2: Load the Boston Housing dataset

(train_data, train_targets), (test_data, test_targets) = boston_housing.load_data()


print("Training data shape:", train_data.shape)

print("Test data shape:", test_data.shape)


# Step 3: Normalize the data (important for Neural Networks)

mean = train_data.mean(axis=0)

std = train_data.std(axis=0)


train_data = (train_data - mean) / std

test_data = (test_data - mean) / std
```

```
# Step 4: Build the Neural Network model
model = keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=(train_data.shape[1],)),
    layers.Dense(64, activation='relu'),
    layers.Dense(1)  # Output layer for regression
])

# Step 5: Compile the model
model.compile(
    optimizer='adam',
    loss='mse',
    metrics=['mae']
)

# Step 6: Train the model
history = model.fit(
    train_data,
    train_targets,
    epochs=100,
    batch_size=16,
    validation_split=0.2,
    verbose=1
)

# Step 7: Evaluate the model
test_loss, test_mae = model.evaluate(test_data, test_targets)
print("\nTest MAE:", test_mae)
```

```
# Step 8: Make predictions
predictions = model.predict(test_data)

print("\nFirst 5 predictions:")
print(predictions[:5])

print("\nActual house prices:")
print(test_targets[:5])

# Step 9: Plot Training and Validation Loss
plt.figure(figsize=(8, 5))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Mean Squared Error')
plt.title('Training vs Validation Loss')
plt.legend()
plt.show()
```

OUTPUT:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/boston_housing.npz
57026/57026 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 1us/step
Training data shape: (404, 13)
Test data shape: (102, 13)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/100
21/21 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3s 32ms/step - loss: 552.2672 - mae: 21.6493 - val_loss: 547.4504 - val_mae: 21.6055
```

Epoch 2/100

21/21 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 454.0424 - mae: 19.4634 - val_loss: 411.1283 - val_mae: 18.3274

Epoch 3/100

21/21 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 305.0794 - mae: 15.3025 - val_loss: 227.8982 - val_mae: 12.7506

Epoch 100/100

21/21 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 6.3196 - mae: 1.7443 - val_loss: 15.0501 - val_mae: 2.6755

4/4 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 16.5313 - mae: 2.7127

Test MAE: 2.920720338821411

4/4 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step

First 5 predictions:

[[8.292983]

 [18.221378]

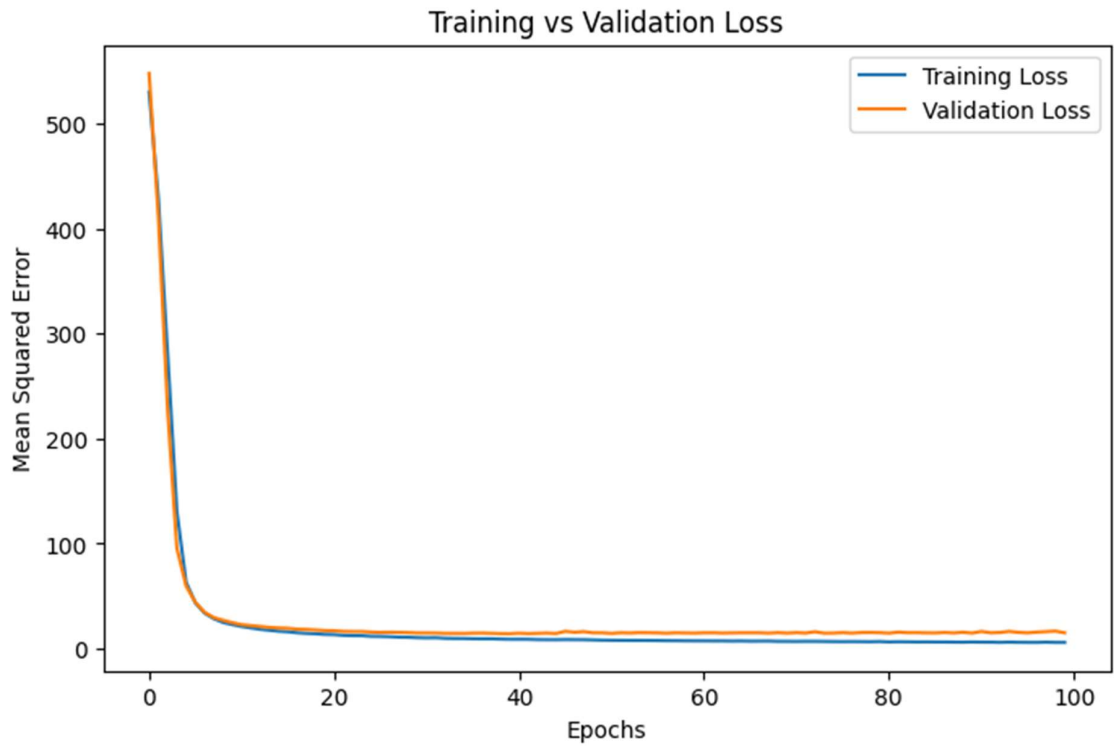
 [21.360199]

 [32.150787]

 [24.964174]]

Actual house prices:

[7.2 18.8 19. 27. 22.2]


Training vs Validation Loss
Training Loss
Validation Loss
Mean Squared Error
500
400
300
200
100
0
0
20
40
60
80
100
Epochs

EXPERIMENT-7

AIM: Use a pre-trained convolution neural network (VGG16) for image classification.

DESCRIPTION:

To use a pre-trained VGG16 convolutional neural network for image classification, you would: load the pre-trained VGG16 model, preprocess your new images to match the format expected by the model, then pass the images through the model to obtain predictions about their class labels; essentially leveraging the features learned from a large dataset (like ImageNet) to classify images from your specific domain by replacing the final fully connected layers with ones tailored to your task, a process called transfer learning.

PROGRAM:

```
# IMPORTS
import tensorflow as tf
from tensorflow.keras.applications import VGG16
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Flatten, Dense, Dropout
from tensorflow.keras.utils import plot_model
from tensorflow.keras.preprocessing.image import load_img
from google.colab import files
import matplotlib.pyplot as plt
#LOAD PRETRAINED VGG16
base_model = VGG16(
    weights='imagenet',
    include_top=False,
    input_shape=(224, 224, 3)
)
# FREEZE BASE MODEL LAYERS
for layer in base_model.layers:
    layer.trainable = False
# ADD CUSTOM CLASSIFICATION LAYERS
x = base_model.output
x = Flatten()(x)
```

```
x = Dense(256, activation='relu')(x)
x = Dropout(0.5)(x)
output = Dense(3, activation='softmax')(x)
# CREATE FINAL MODEL
model = Model(
    inputs=base_model.input,
    outputs=output
)
# COMPILE MODEL
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
# MODEL SUMMARY
model.summary()
#  VISUALIZE MODEL ARCHITECTURE
plot_model(
    model,
    show_shapes=True,
    show_layer_names=True
)
#UPLOAD IMAGE
uploaded = files.upload()
# LOAD AND DISPLAY IMAGE
img = load_img('dog1.jpg', target_size=(224, 224))
plt.imshow(img)
plt.axis('off')
plt.show()
```

OUTPUT:

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5

58889256/58889256 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 0us/step

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 224, 224, 3)	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1,792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36,928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73,856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147,584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295,168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590,080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590,080
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 14, 14, 512)	0

block5_conv1 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, 7, 7, 512)	0
flatten (Flatten)	(None, 25088)	0
dense (Dense)	(None, 256)	6,422,784
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 3)	771

Total params: 21,138,243 (80.64 MB)
Trainable params: 6,423,555 (24.50 MB)
Non-trainable params: 14,714,688 (56.13 MB)

Choose Files dog1.jpg

dog1.jpg(image/jpeg) - 10350 bytes, last modified: 2/28/2026 - 100% done
Saving dog1.jpg to dog1.jpg

EXPERIMENT-9

AIM: Implement word embeddings for IMDB dataset.

DESCRIPTION:
Word embedding is essential in natural language processing with deep learning. This technique allows the network to learn about the meaning of the words. In this post, we classify movie reviews in the IMDB dataset as positive or negative, and provide a visual illustration of embedding.

Today is to train a neural network to find out whether some text is globally positive or negative, a task called sentiment analysis.

The first layer of our neural network will perform an operation called word embedding, which is essential in NLP with deep learning.

The IMDB database: We will work with the IMDB dataset, which contains 25,000 movie reviews from IMDB. Each review is labeled as positive or negative from the rating provided by users together with their reviews.

PROGRAM:

```
# Set random seeds for reproducibility

from numpy.random import seed

seed(0xdeadbeef)


import tensorflow as tf

tf.random.set_seed(0xdeadbeef)


from tensorflow import keras

import matplotlib.pyplot as plt


# Load IMDB dataset

imdb = keras.datasets.imdb

num_words = 20000


(train_data, train_labels), (test_data, test_labels) = imdb.load_data(

    seed=1,
```

```
    num_words=num_words
)
print(train_data[0])
print("Label:", train_labels[0])

# Get word index dictionary
vocabulary = imdb.get_word_index()

# Shift indices by 3 (reserve special tokens)
vocabulary = {k: (v + 3) for k, v in vocabulary.items()}
vocabulary["<PAD>"] = 0
vocabulary["<START>"] = 1
vocabulary["<UNK>"] = 2
vocabulary["<UNUSED>"] = 3

# Create reverse dictionary (number → word)
index = dict((value, key) for (key, value) in vocabulary.items())

# Function to convert numbers back to words
def decode_review(text):
    return ' '.join([index.get(i, '?') for i in text])

# Print decoded review
print(decode_review(train_data[0]))

# Pad sequences to same length (256 words)
train_data = keras.preprocessing.sequence.pad_sequences(
    train_data,
    value=vocabulary["<PAD>"],
    padding='post',
```

```
    maxlen=256
)

test_data = keras.preprocessing.sequence.pad_sequences(
    test_data,
    value=vocabulary["<PAD>"],
    padding='post',
    maxlen=256
)
# Build the model
model = keras.Sequential()

# Embedding layer (word → vector)
model.add(keras.layers.Embedding(len(vocabulary), 2, input_length=256))

# Flatten output
model.add(keras.layers.Flatten())

# Add dropout to prevent overfitting
model.add(keras.layers.Dropout(0.5))

# Hidden dense layer
model.add(keras.layers.Dense(5))

# Output layer (binary classification)
model.add(keras.layers.Dense(1, activation='sigmoid'))
model.summary()

# Compile model
model.compile(
```

```
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# Train model
history = model.fit(
    train_data,
    train_labels,
    epochs=5,
    batch_size=100,
    validation_data=(test_data, test_labels),
    verbose=1
)

# Plot accuracy graph
def plot_accuracy(history):
    acc = history.history['accuracy']
    val_acc = history.history['val_accuracy']
    epochs = range(len(acc))
    plt.plot(epochs, acc, label='Training Accuracy')
    plt.plot(epochs, val_acc, label='Validation Accuracy')
    plt.title('Accuracy')
    plt.xlabel('Epoch')
    plt.legend()
    plt.show()

plot_accuracy(history)
```

OUTPUT:

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz

17464789/17464789 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 0us/step

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb_word_index.json

1641221/1641221 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 0us/step

warnings.warn(

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
flatten (Flatten)	?	0 (unbuilt)
dropout (Dropout)	?	0
dense (Dense)	?	0 (unbuilt)
dense_1 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

Epoch 1/5

250/250 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.5229 - loss: 0.6914 - val_accuracy: 0.7464 - val_loss: 0.6104

Epoch 2/5

250/250 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 0.7699 - loss: 0.5213 - val_accuracy: 0.8608 - val_loss: 0.3541

Epoch 3/5

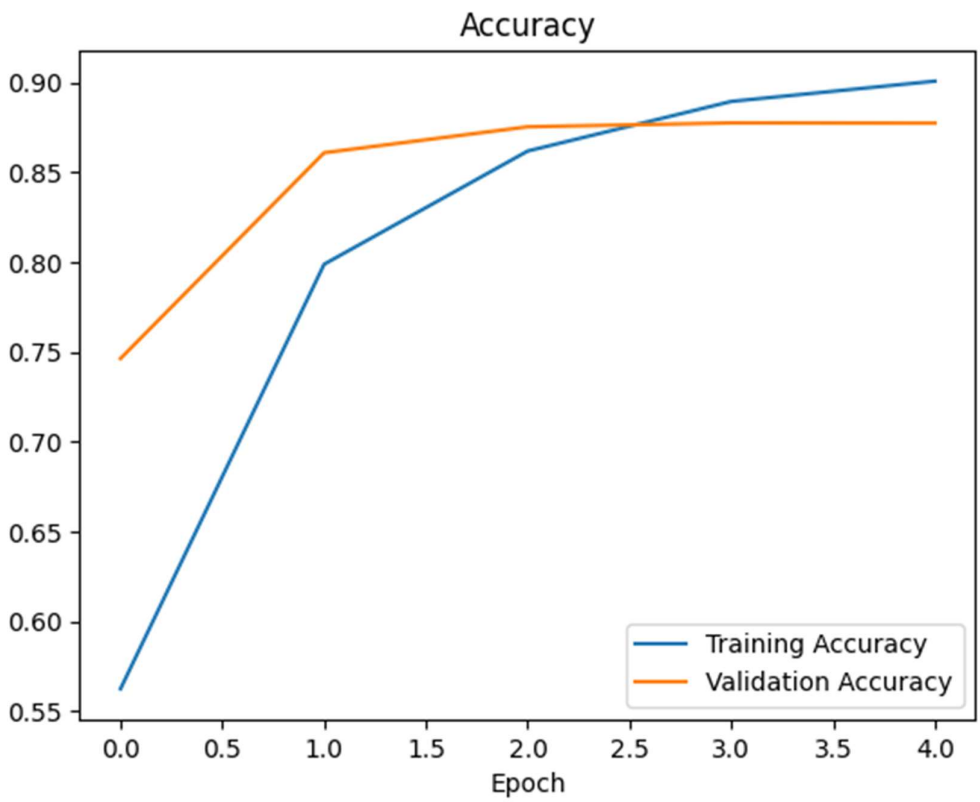
250/250 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 0.8528 - loss: 0.3383 - val_accuracy: 0.8753 - val_loss: 0.3088

Epoch 4/5

250/250 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - accuracy: 0.8867 - loss: 0.2767 - val_accuracy: 0.8774 - val_loss: 0.2966

Epoch 5/5

250/250 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4s 8ms/step - accuracy: 0.9008 - loss: 0.2395 - val_accuracy: 0.8773 - val_loss: 0.2965


Accuracy
0.90
0.85
0.80
0.75
0.70
0.65
0.60
0.55
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Epoch
Training Accuracy
Validation Accuracy